GFAz: Order-of-Magnitude Pangenome Analytics by Computing over Compression

Compress the pangenome **84× smaller** at **GiB/s** throughput. Then run the analyses **up to 619× faster**, in **up to 25× less memory**.

Taolue Yang Youyuan Liu
Bo Jiang Xinghua Shi Sian Jin
Department of Computer & Information Sciences
Temple University

T2T Face-to-Face 2026 · ICS '26

The files are enormous

Traversals are over **90%** of a GFA file. HPRC v2 whole-genome graphs reach **369 GB** of text; cohort-scale graphs are already in the terabytes.

Generic codecs miss the structure

gzip and Zstd match over local byte windows, so cross-haplotype repeats go unexploited. Specialized tools drop optional fields or run at a few MB/s.

Analysis pays the cost twice

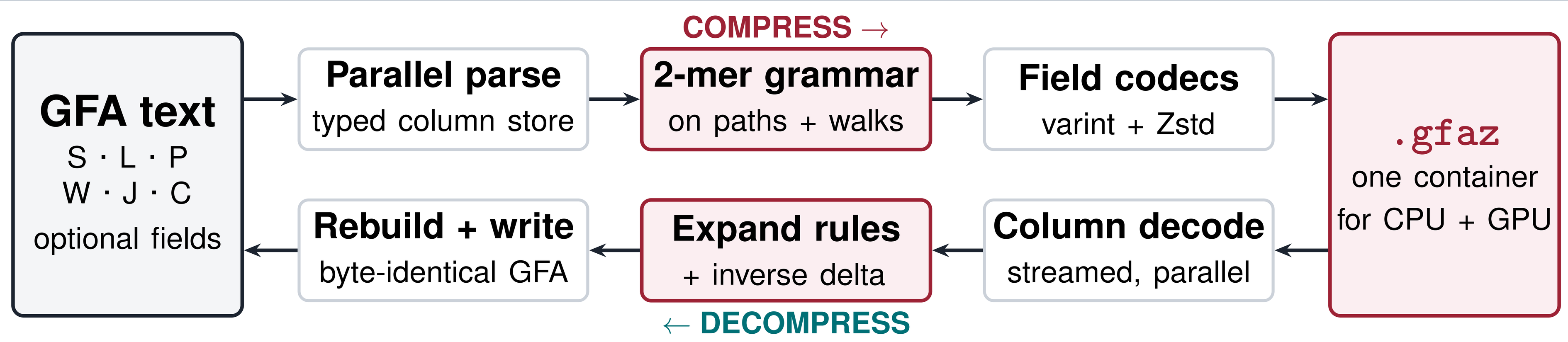
`vg`, `odgi` and Panacus must decompress *and* rebuild the whole graph before answering one query. That costs hundreds of GB of RAM.

84 ×	1.36 GiB/s	5.43 GiB/s	619 ×	25 ×
smaller on disk	**compression**	**decompression**	**faster analysis**	**less memory**
369 GB → 4.5 GB	CPU · 32 threads	CPU · 32 threads	vs. `odgi` / `vg` / Panacus	peak RSS, up to

1 Compression engine

lossless · every GFA record type · linear time

A lossless round trip through one container



Both directions are **fully parallel** (OpenMP; a CUDA backend shares the same format). Columns are varint- and bit-packed before Zstd. Optional fields and record order survive intact, so `.gfaz` *replaces* the GFA rather than shadowing it.

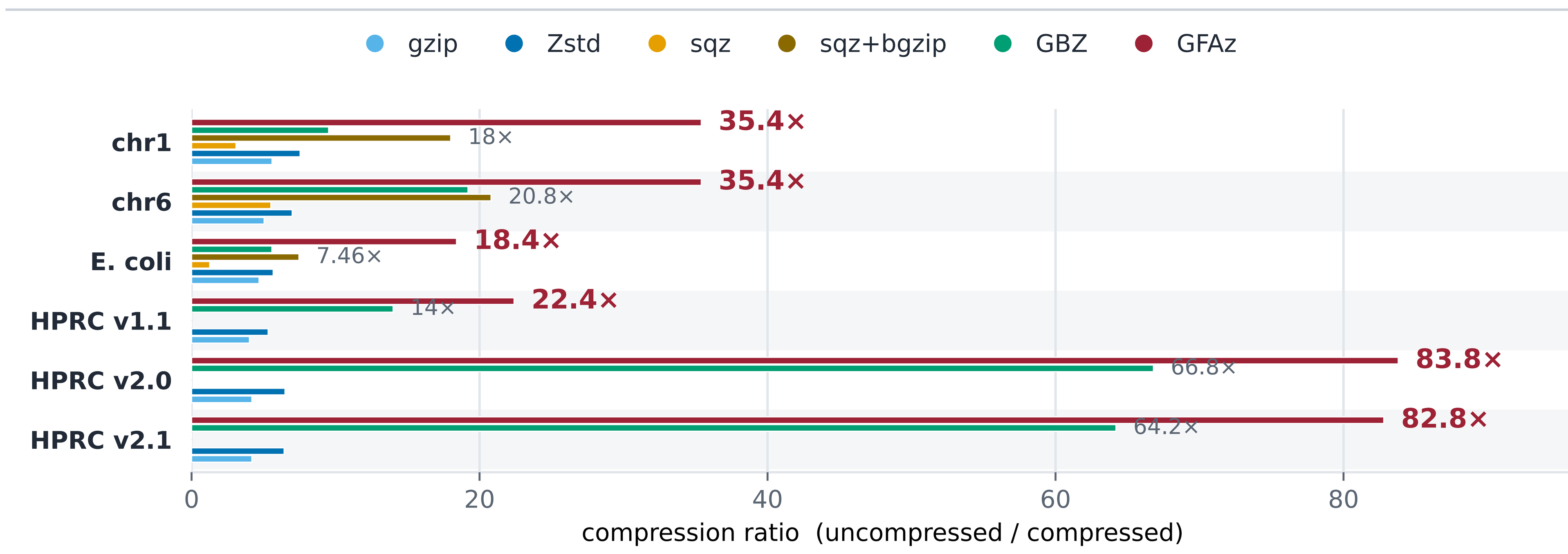
The idea: a recursive grammar of 2-mers

Haplotypes repeat each other. GFAz replaces the most frequent *adjacent pair* of symbols with a rule, then repeats. Later rounds reuse earlier rules, so each round captures longer repeats.

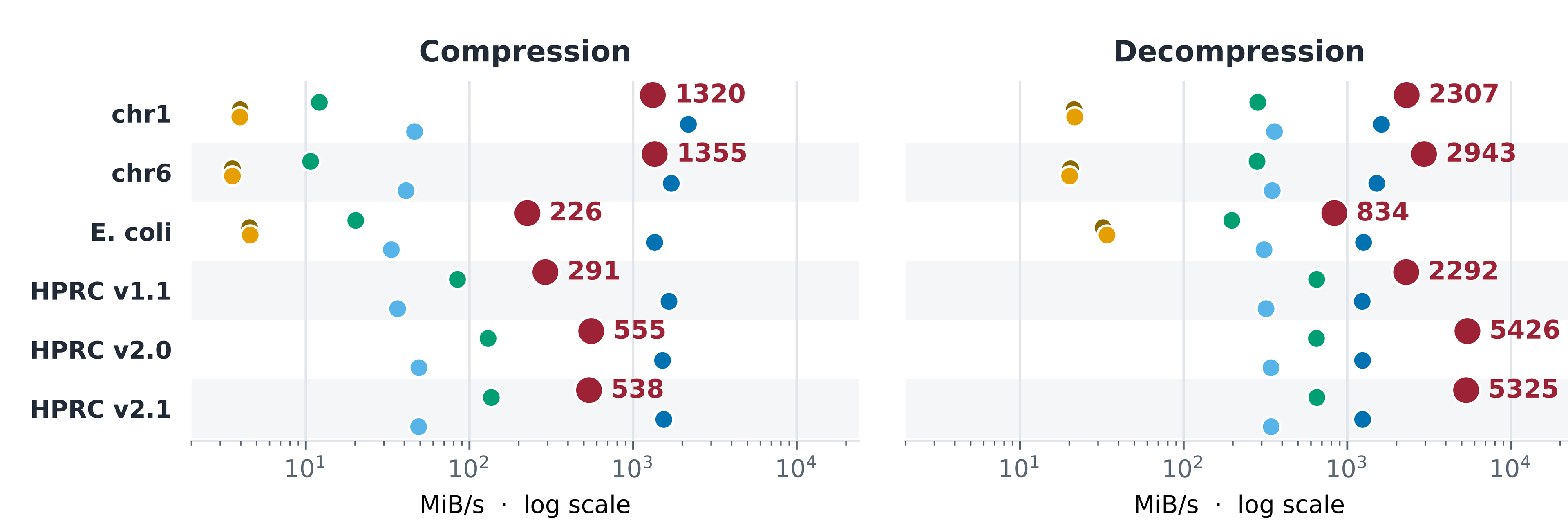
Input	no rules yet	12 7 12 7 12 7 12 7 31
Round 1	$R_1 \leftarrow (12, 7)$	R_1 R_1 R_1 R_1 31
Round 2	$R_2 \leftarrow (R_1, R_1)$	R_2 R_2 31
Round 3	$R_3 \leftarrow (R_2, R_2)$	R_3 31

Each round is one count-and-rewrite scan: $O(N)$ **per round, fixed round count**, so compression is linear and parallel across chunks. Paths and walks share one rulebook. Orientations are canonicalized, so a forward repeat also matches its reverse complement.

CPU performance against five baselines



GFAz compresses better than every baseline on all six graphs, reaching 84× on HPRC v2.0: 20× past gzip, 13× past Zstd.

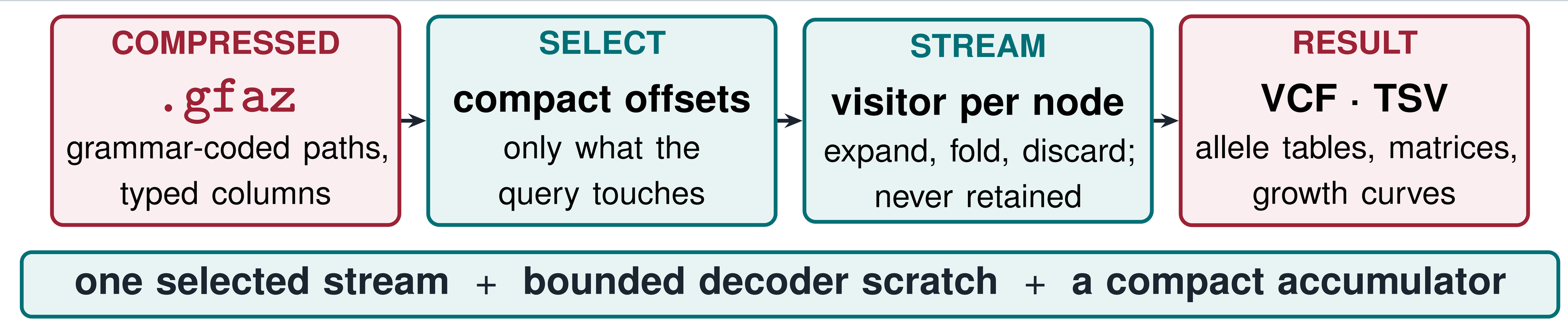


End-to-end CLI wall time, 32 threads; same legend as above. **GFAz decompresses fastest on every graph** (up to 5.4 GiB/s) and compresses 30–90× faster than gzip, GBZ and sqz. Only Zstd compresses faster, at 5–13× worse ratio.

2 Compute engine

run the analysis on the compressed container

The graph is never rebuilt



UNUSED STREAMS STAY COMPRESSED · THE FULL GFA IS NEVER MATERIALIZED

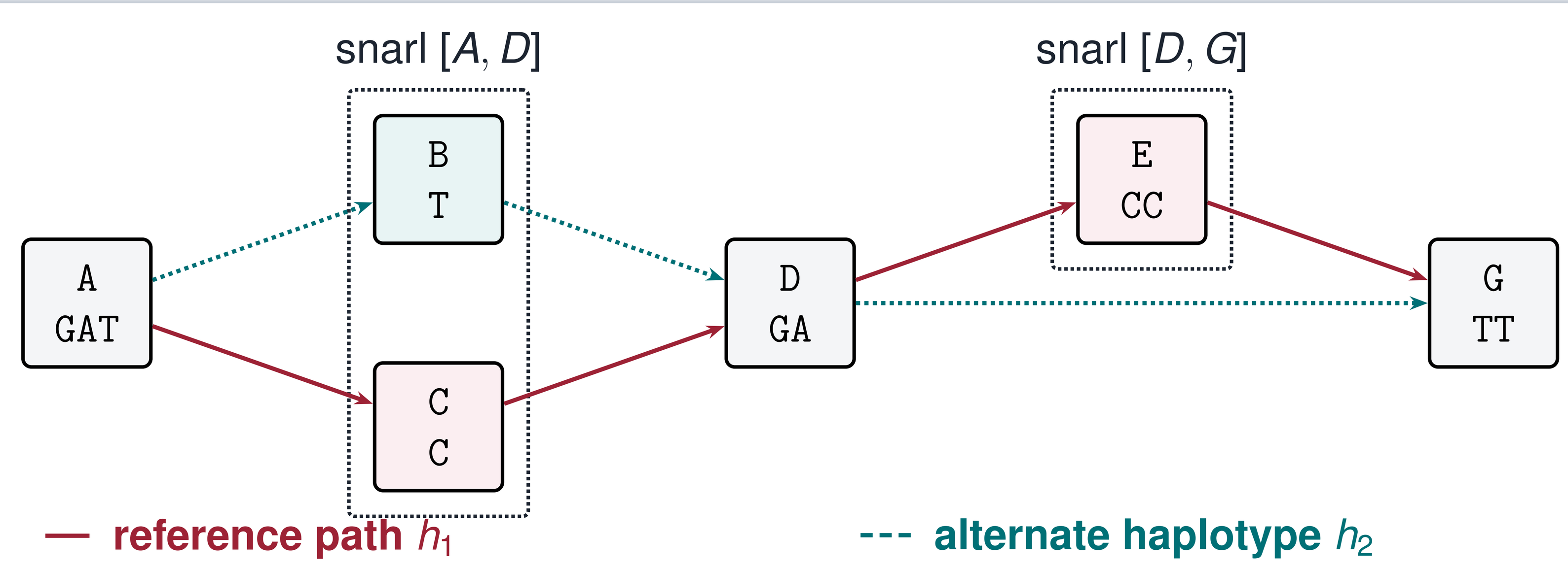
Most analyses only *read* traversals, folding node visits into a small accumulator. GFAz runs that fold on the grammar-compressed columns, so **peak memory stays below the original GFA**.

Six analyses, one decoder

GFAz command	Reproduces	Passes	Accumulator
`deconstruct`	`vg deconstruct`	1 (+topo)	per-site allele table
`growth`	Panacus	1	coverage → histogram
`depth`	`odgi depth`	1	per-node counters
`stats`	`odgi stats`	0	metadata totals
`pav`	`odgi pav`	3	node→groups CSR
`similarity`	`odgi similarity`	2 (+join)	node→groups CSR

Only the accumulator column changes. **Adding an analysis means writing one fold function**, not a new engine: all six reuse the same decoder, memory model and parallel schedule.

`deconstruct`: GFA → VCF with no graph



#CHROM	POS	REF	ALT	INFO	h1	h2
chr1	4	C	T	AC=1;AN=2;AF=0.5	0	1
chr1	6	ACC	A	AC=1;AN=2;AF=0.5	0	1

Phase 1 recovers the snarls from *topology alone*: a biconnected decomposition in $O(V+E)$ that decodes zero haplotypes. **Phase 2** streams each haplotype once through a boundary state machine; a reverse boundary pair normalizes inversions. No graph object, no general snarl index.

Same answers, orders of magnitude cheaper

Analysis	Graph	Baseline	GFAz	Speedup	Mem.
`deconstruct` *vs.* `vg`	chr1	805 s / 30.3 GB	11.3 s / 8.3 GB	**71×**	**3.7×**
	HGSVC3	132.9 min / 397 GB	8.7 min / 51.9 GB	**15×**	**7.6×**
`growth` *vs.* Panacus	chr1	18.3 s / 6.16 GB	0.74 s / 0.66 GB	**25×**	**9.3×**
	v2.0	245 min / 327 GB	39.8 s / 12.9 GB	**369×**	**25×**
`pav` *vs.* `odgi`	chr1	3499 s / 31.9 GB	11.7 s / 9.5 GB	**299×**	**3.4×**
	chr6	4759 s / 19.4 GB	7.8 s / 6.4 GB	**619×**	**3.0×**

Wall clock / peak RSS at 16 threads. **The outputs match the baselines:** `growth` reproduces Panacus' curve exactly, `deconstruct` agrees with `vg` to within 0.2% of records, `pav` matrices are structurally identical to `odgi`'s. On the 358/369 GB HPRC graphs `vg` and `odgi` *cannot load the input at all*; **GFAz answers from a 4.5 GB container**.

code

Open source: `github.com/babyplutokurt/GFAz` · **Contact:** `taolue.yang@temple.edu`

Compression engine: Yang, Liu, Jiang, Shi, Jin. *GFAz: State-of-the-Art Graphical Fragment Assembly Compression.* ICS '26. `doi:10.1145/3797905.3807870`
Compute engine: Yang, Liu, Jiang, Shi, Jin. *GFAz: Order-of-Magnitude Pangenome Analytics by Computing over Compression.* In submission, 2026.
Acknowledgments: We thank the Human Pangenome Reference Consortium (BioProject PRJNA730823) and its funder, NHGRI. Supported in part by NIH award R01GM157443.

ICS paper